PYTHON PRACTICAL RECORD

Program Names • Algorithms • Symbol-Based Flowcharts • Pseudocode • Test Cases • Output Screenshots

Standard symbols: Oval = Start/End; Parallelogram = Input/Output; Rectangle = Process; Diamond = Decision. Arrows show the direction of the program flow.

Prepared in simple English from the provided program screenshots. Each program includes a standard-symbol flowchart and its output screenshot.

Contents

1. 1. Shopping Bill Calculator
2. 2. Retail Billing System
3. 3. Exam Result Calculator
4. 4. Attendance Recovery Calculator
5. 5. ATM Transaction Menu
6. 6. Performance Trend Analyzer
7. 7. Shopping Budget Checker
8. 8. Numeric Pattern Generator
9. 9. Text Compression (Run-Length Encoding)

1. Shopping Bill Calculator

Aim: To calculate the total cost of three items using their prices and quantities.

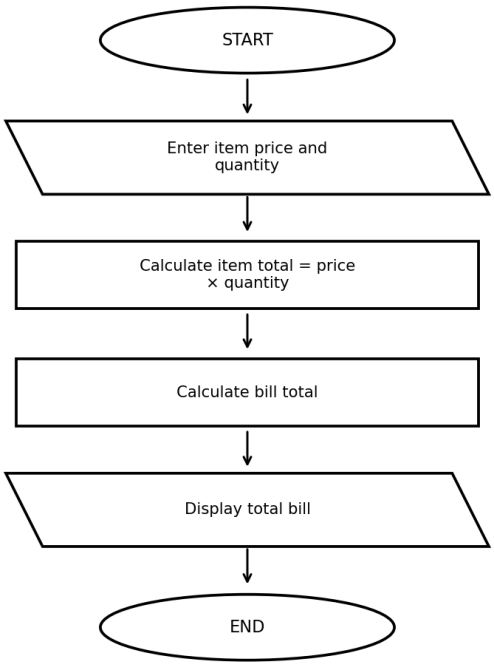
Algorithm

10. Start the program.
11. Set total = 0.
12. Enter the price and quantity for each of 3 items.
13. If price or quantity is negative, display an error and stop.
14. Add price × quantity to total for each item.
15. Display the total bill and stop.

Python Program

```
total = 0.0
for item in range(1, 4):
    price = float(input(f"Enter price of item {item}: ₹"))
    quantity = int(input(f"Enter quantity of item {item}: "))
    if price < 0 or quantity < 0:
        print("Error: price and quantity cannot be negative.")
        raise SystemExit
    total += price * quantity
print(f"Total shopping bill: ₹{total:.2f}")
```

Flowchart with Standard Symbols



Oval = Start/End | Parallelogram = Input/Output | Rectangle = Process | Diamond = Decision

Pseudocode

```
START
total = 0
REPEAT 3 TIMES:
    INPUT price, quantity
```

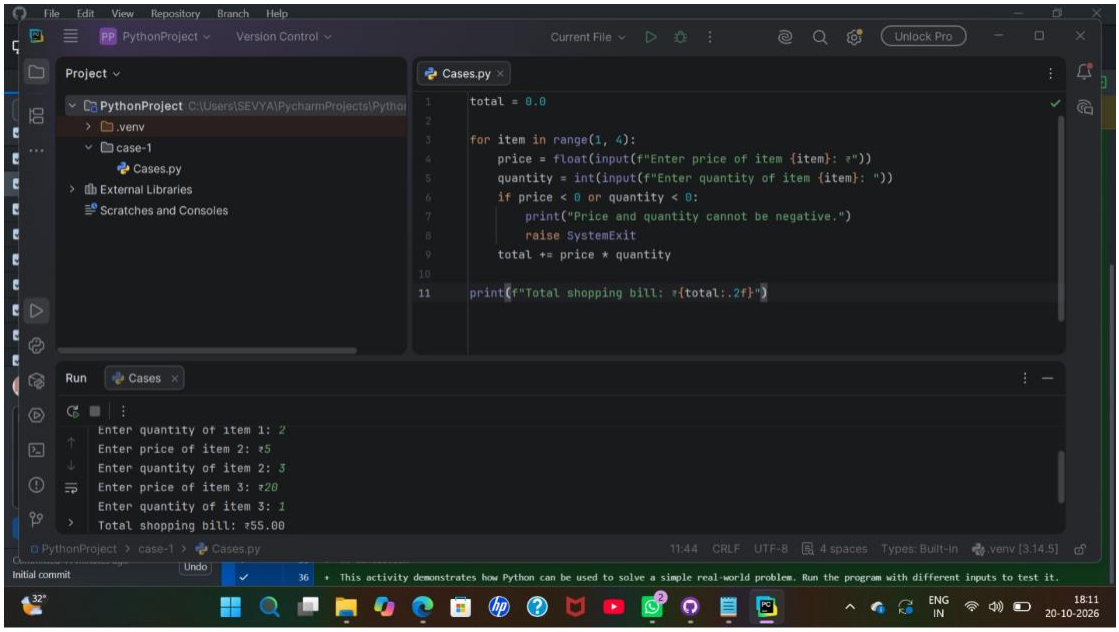
```
    IF price < 0 OR quantity < 0, SHOW error and STOP
    total = total + price * quantity
DISPLAY total bill
STOP
```

Test Cases

Test	Input	Expected result
Valid values	Enter prices and quantities for 3 items	Total bill is displayed
Negative value	Enter a negative price or quantity	Error message appears

Output Screenshot

The screenshot shows the total shopping bill as ₹55.00.



2. Retail Billing System

Aim: To prepare a bill for items and calculate the grand total.

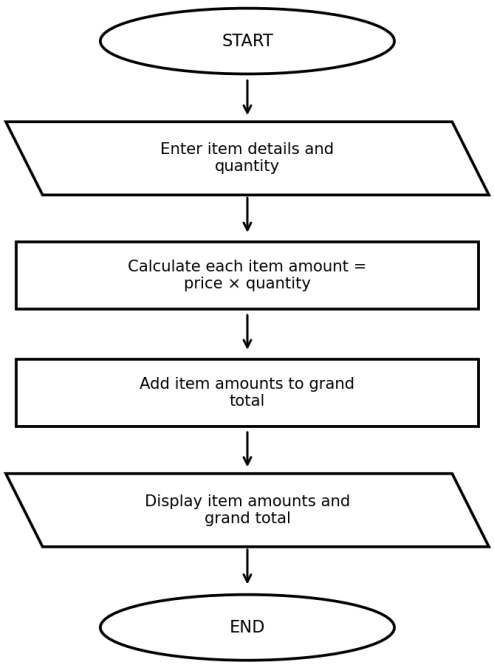
Algorithm

16. Enter the number of items.
17. Set grand_total = 0.
18. For each item, enter its name, quantity and price.
19. Calculate subtotal = quantity × price.
20. Add subtotal to grand_total and display the item details.
21. Display grand_total and stop.

Python Program

```
n = int(input("Enter number of items: "))
if n <= 0:
    print("Error: number of items must be positive.")
    raise SystemExit
grand_total = 0.0
for i in range(1, n + 1):
    name = input(f"Enter name of item {i}: ")
    quantity = int(input("Enter quantity: "))
    price = float(input("Enter unit price: ₹"))
    if quantity < 0 or price < 0:
        print("Error: quantity and price cannot be negative.")
        raise SystemExit
    subtotal = quantity * price
    grand_total += subtotal
    print(f"{name}: {quantity} x ₹{price:.2f} = ₹{subtotal:.2f}")
print(f"Grand Total: ₹{grand_total:.2f}")
```

Flowchart with Standard Symbols



Oval = Start/End | Parallelogram = Input/Output | Rectangle = Process | Diamond = Decision

Pseudocode

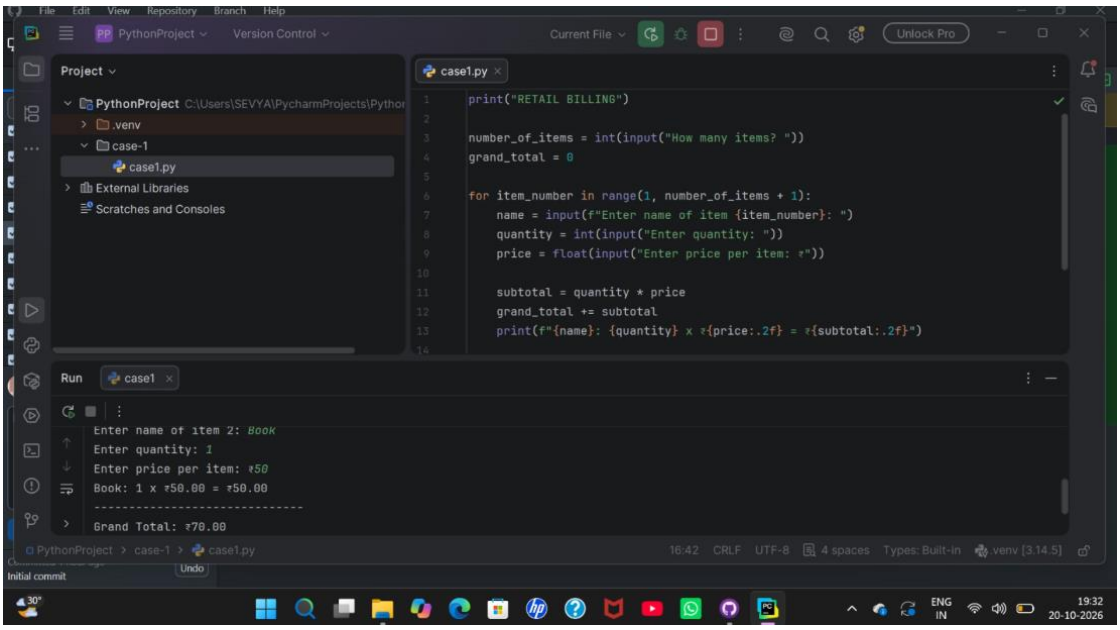
```
START
INPUT number of items
grand_total = 0
FOR each item:
    INPUT name, quantity, price
    subtotal = quantity * price
    grand_total = grand_total + subtotal
    DISPLAY item and subtotal
DISPLAY grand_total
STOP
```

Test Cases

Test	Input	Expected result
One book	Book, quantity 1, price ₹50	Book subtotal = ₹50.00
Several items	Enter two or more items	Each subtotal and grand total are displayed

Output Screenshot

The visible output includes Book: 1 × ₹50.00 = ₹50.00 and Grand Total: ₹70.00.



3. Exam Result Calculator

Aim: To calculate average marks, assign a grade and decide whether the student passes.

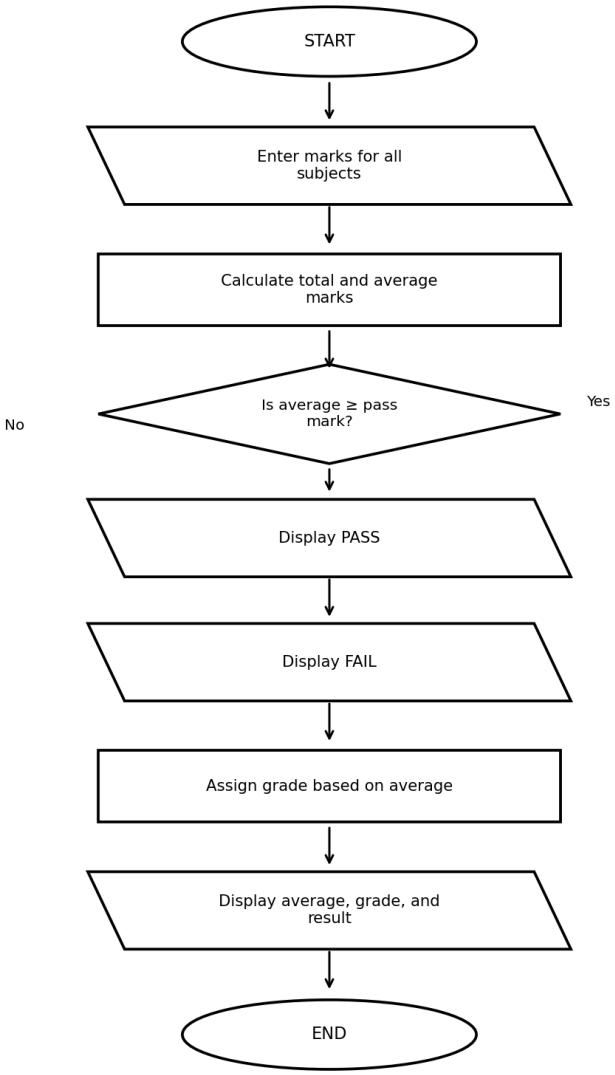
Algorithm

22. Enter the number of subjects.
23. Set total = 0 and pass = YES.
24. Enter marks for each subject.
25. If marks are outside 0–100, display an error and stop.
26. Add marks to total. If any mark is below 35, set pass = NO.
27. Calculate average, find the grade and display the result.

Python Program

```
n = int(input("Enter number of subjects: "))
if n <= 0:
    print("Error: enter at least one subject.")
    raise SystemExit
total = 0
passed = True
for i in range(1, n + 1):
    marks = float(input(f"Enter marks for subject {i} (0-100): "))
    if marks < 0 or marks > 100:
        print("Error: marks must be between 0 and 100.")
        raise SystemExit
    total += marks
    if marks < 35:
        passed = False
average = total / n
if average >= 90:
    grade = "A"
elif average >= 75:
    grade = "B"
elif average >= 60:
    grade = "C"
elif average >= 35:
    grade = "D"
else:
    grade = "F"
print(f"Average: {average:.2f}")
print(f"Grade: {grade}")
print("Result: Pass" if passed else "Result: Fail")
```

Flowchart with Standard Symbols



Oval = Start/End | Parallelogram = Input/Output | Rectangle = Process | Diamond = Decision

Pseudocode

```
START
INPUT number of subjects
total = 0; pass = YES
FOR each subject:
    INPUT marks
    IF marks < 0 OR marks > 100, SHOW error and STOP
    total = total + marks
    IF marks < 35, pass = NO
average = total / number of subjects
FIND grade
DISPLAY average, grade and result
STOP
```

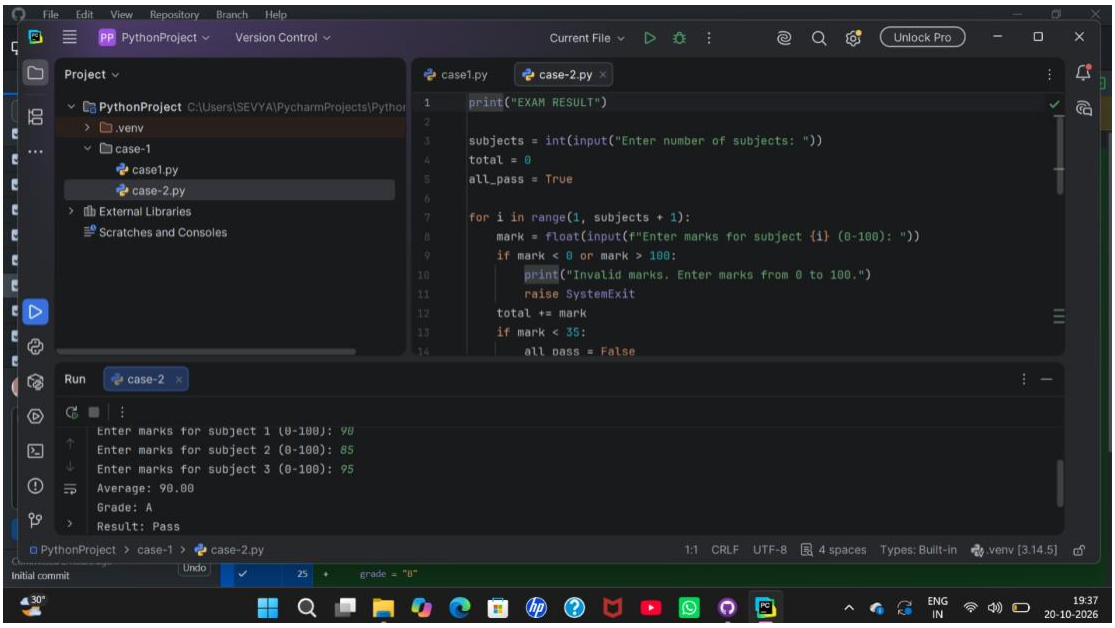
Test Cases

Test	Input	Expected result

Passing marks	90, 85, 95	Average 90.00, Grade A, Pass
Low mark	One mark is below 35	Result is Fail
Invalid mark	Enter 105	Invalid marks message appears

Output Screenshot

The screenshot shows Average: 90.00, Grade: A, Result: Pass.



4. Attendance Recovery Calculator

Aim: To find how many extra classes are needed to reach the target attendance.

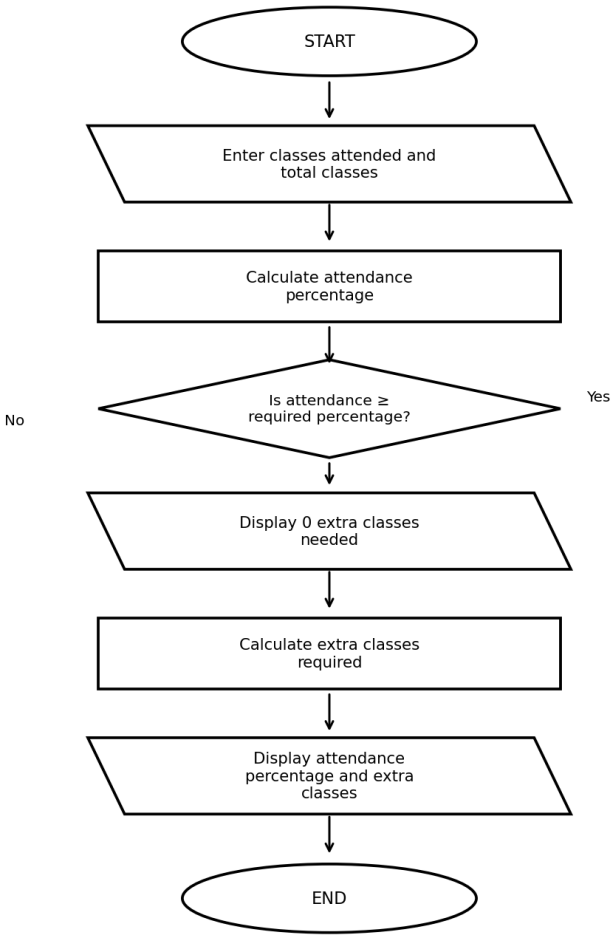
Algorithm

28. Enter attended classes, total classes and target percentage.
29. Check that all values are valid.
30. Calculate current attendance percentage.
31. Set extra = 0.
32. While attendance is below target, add one attended class and one total class; increase extra by one.
33. Display extra classes and final attendance.

Python Program

```
attended = int(input("Enter classes attended: "))
total = int(input("Enter total classes held: "))
target = float(input("Enter target attendance percentage: "))
if total <= 0 or attended < 0 or attended > total or not 0 <= target <= 100:
    print("Error: enter valid attendance values.")
    raise SystemExit
extra = 0
while attended / total * 100 < target:
    attended += 1
    total += 1
    extra += 1
print(f"Extra classes needed: {extra}")
print(f"Final attendance: {attended}/{total} = {attended / total * 100:.2f}%")
```

Flowchart with Standard Symbols



Oval = Start/End | Parallelogram = Input/Output | Rectangle = Process | Diamond = Decision

Pseudocode

```
START
INPUT attended, total, target
IF values are invalid, SHOW error and STOP
current = attended / total * 100
extra = 0
WHILE current < target:
    attended = attended + 1
    total = total + 1
    extra = extra + 1
    current = attended / total * 100
DISPLAY extra and final attendance
STOP
```

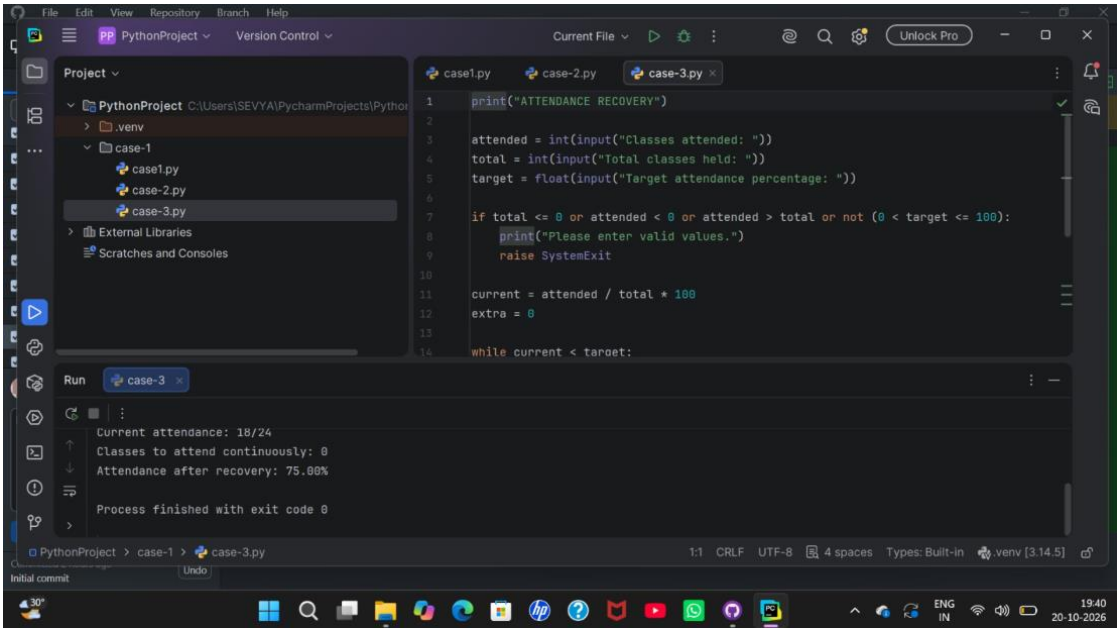
Test Cases

Test	Input	Expected result
At target	18 attended, 24 total, target 75%	Extra classes = 0; attendance = 75%
Below target	Enter attendance below target	Extra classes counted until target is met

Invalid values	Total classes = 0	Error message appears

Output Screenshot

The screenshot shows current attendance 18/24, 0 extra classes and 75.00% after recovery.



5. ATM Transaction Menu

Aim: To check balance, deposit money and withdraw money using a menu.

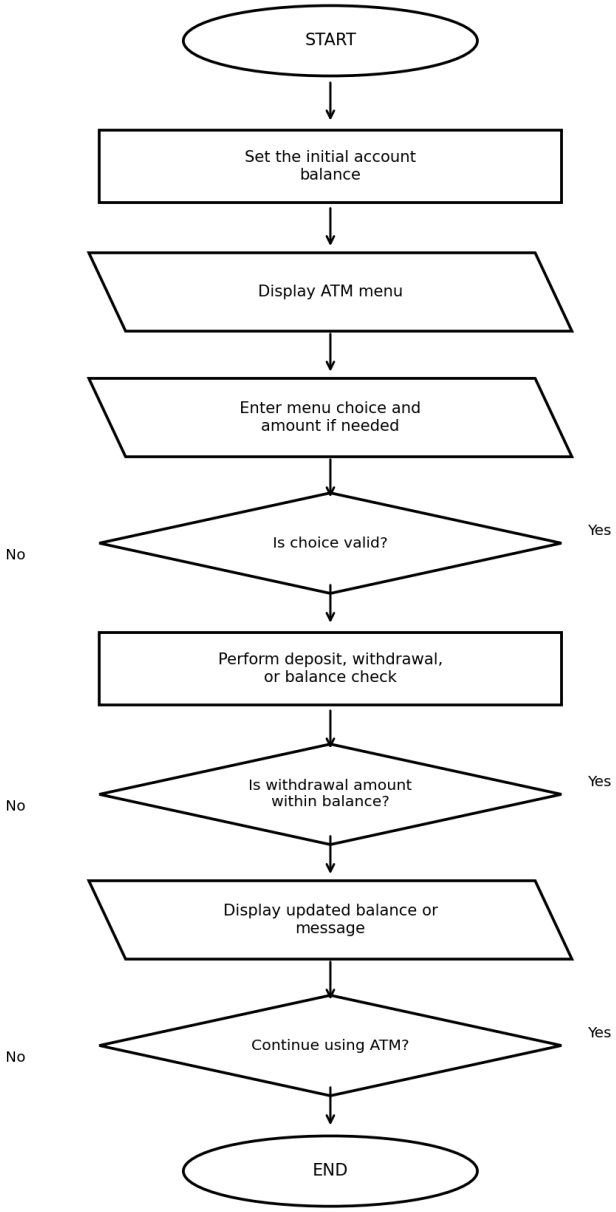
Algorithm

34. Set balance = ₹5000.
35. Display the menu and read the user's choice.
36. If choice is 1, display balance.
37. If choice is 2, add a positive deposit to balance.
38. If choice is 3, withdraw only a valid amount within the balance.
39. If choice is 4, exit; otherwise display an error and repeat the menu.

Python Program

```
balance = 5000.0
while True:
    print("\n1. Check balance\n2. Deposit\n3. Withdraw\n4. Exit")
    choice = input("Enter choice: ")
    if choice == "1":
        print(f"Available balance: ₹{balance:.2f}")
    elif choice == "2":
        amount = float(input("Enter deposit amount: ₹"))
        if amount > 0:
            balance += amount
            print(f"New balance: ₹{balance:.2f}")
        else:
            print("Error: amount must be positive.")
    elif choice == "3":
        amount = float(input("Enter withdrawal amount: ₹"))
        if amount <= 0:
            print("Error: amount must be positive.")
        elif amount > balance:
            print("Error: insufficient balance.")
        else:
            balance -= amount
            print(f"New balance: ₹{balance:.2f}")
    elif choice == "4":
        print("Thank you. Goodbye!")
        break
    else:
        print("Invalid choice. Select 1, 2, 3 or 4.")
```

Flowchart with Standard Symbols



Oval = Start/End | Parallelogram = Input/Output | Rectangle = Process | Diamond = Decision

Pseudocode

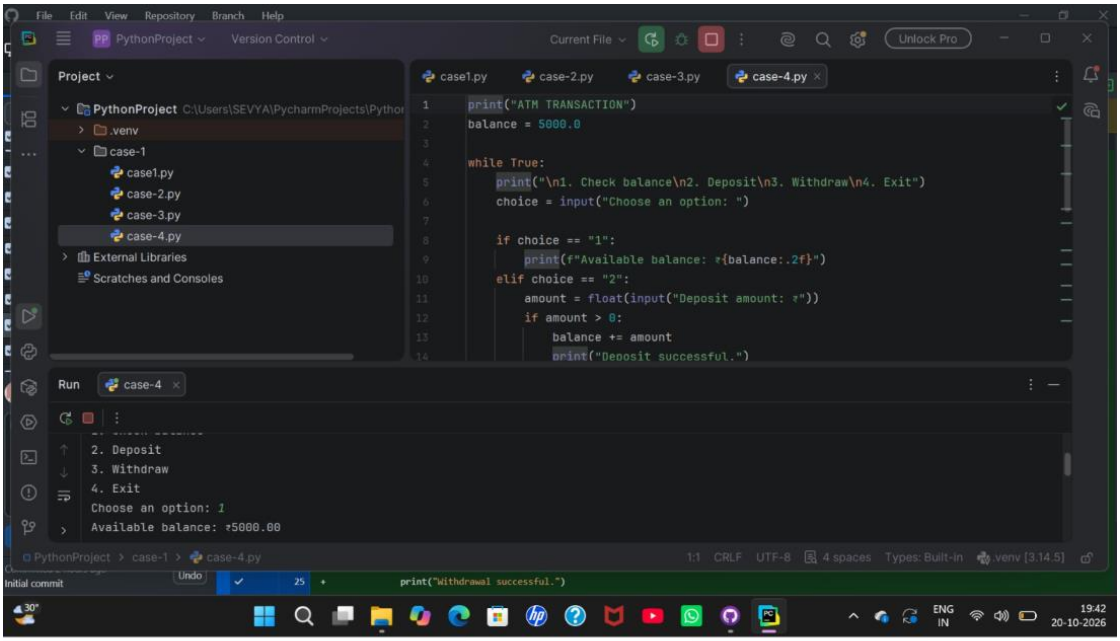
```
START
balance = 5000
REPEAT:
    DISPLAY menu: Balance, Deposit, Withdraw, Exit
    INPUT choice
    IF choice = 1, DISPLAY balance
    IF choice = 2, INPUT amount and add if positive
    IF choice = 3, INPUT amount and withdraw if valid
    IF choice = 4, EXIT
UNTIL exit
STOP
```

Test Cases

Test	Input	Expected result
Check balance	Choose 1	Balance ₹5000.00 is displayed
Deposit	Choose 2 and enter a positive amount	Balance increases
Withdraw	Choose 3 and enter a valid amount	Balance decreases
Exit	Choose 4	Program ends

Output Screenshot

The screenshot shows Available balance: ₹5000.00.



6. Performance Trend Analyzer

Aim: To calculate the average test marks and identify the performance trend.

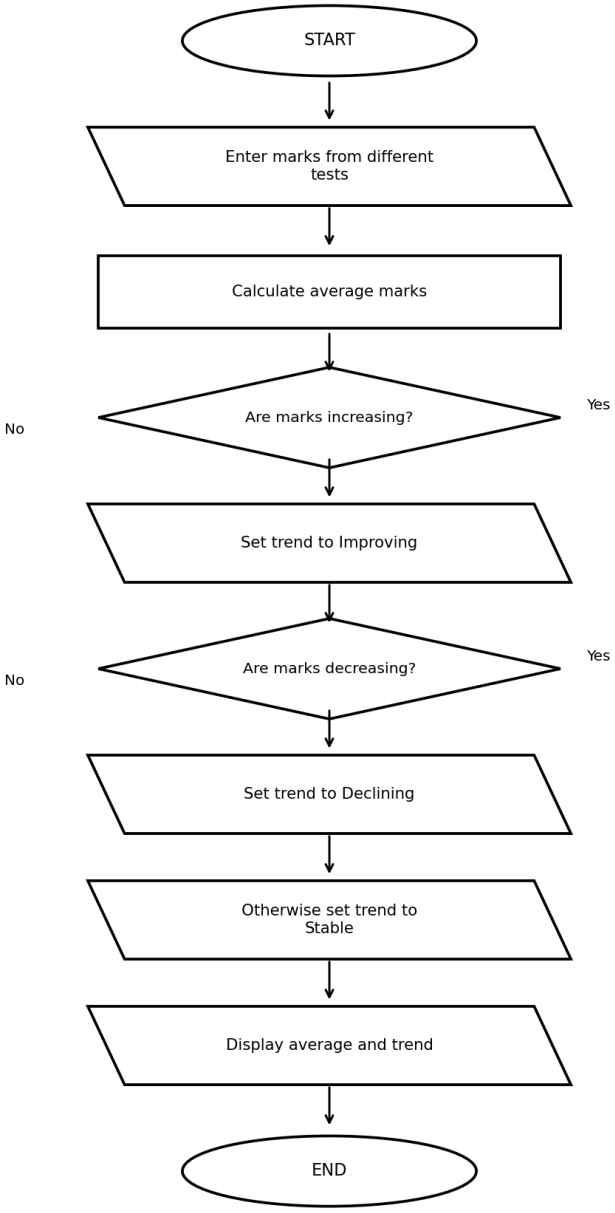
Algorithm

40. Enter the number of tests.
41. If the number is zero or negative, display a message and stop.
42. Enter and store the marks for each test.
43. Calculate average = sum of marks ÷ number of tests.
44. Compare marks to identify the trend.
45. Display average and trend.

Python Program

```
n = int(input("Enter number of tests: "))
if n <= 0:
    print("Please enter at least one test.")
    raise SystemExit
marks = []
for i in range(1, n + 1):
    marks.append(float(input(f"Enter marks for test {i}: ")))
average = sum(marks) / n
if len(marks) < 2:
    trend = "Not enough tests to determine trend"
elif all(marks[i] > marks[i - 1] for i in range(1, n)):
    trend = "Improving"
elif all(marks[i] < marks[i - 1] for i in range(1, n)):
    trend = "Declining"
else:
    trend = "Mixed or unchanged"
print(f"Average: {average:.2f}")
print(f"Trend: {trend}")
```

Flowchart with Standard Symbols



Oval = Start/End | Parallelogram = Input/Output | Rectangle = Process | Diamond = Decision

Pseudocode

```
START
INPUT number of tests
IF number <= 0, SHOW error and STOP
INPUT marks for each test
average = sum of marks / number of tests
CHECK marks to find trend
DISPLAY average and trend
STOP
```

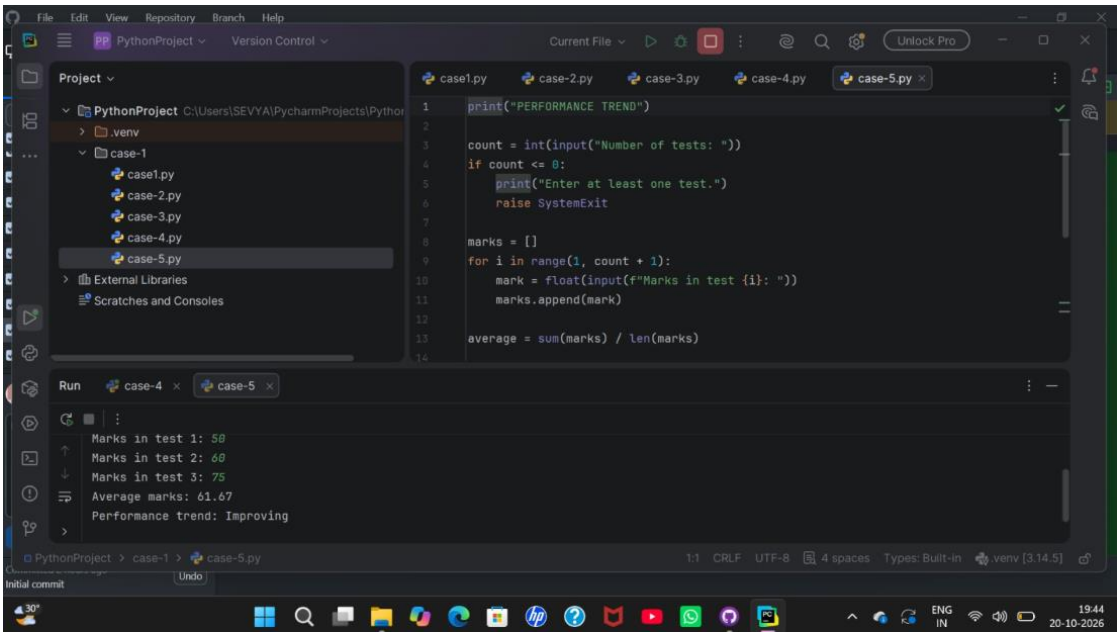
Test Cases

Test	Input	Expected result

Improving marks	50, 60, 75	Average 61.67; trend Improving
No tests	Enter 0	Message asks for at least one test

Output Screenshot

The screenshot shows marks 50, 60 and 75, average 61.67 and trend Improving.



7. Shopping Budget Checker

Aim: To calculate total item cost and check whether it is within the budget.

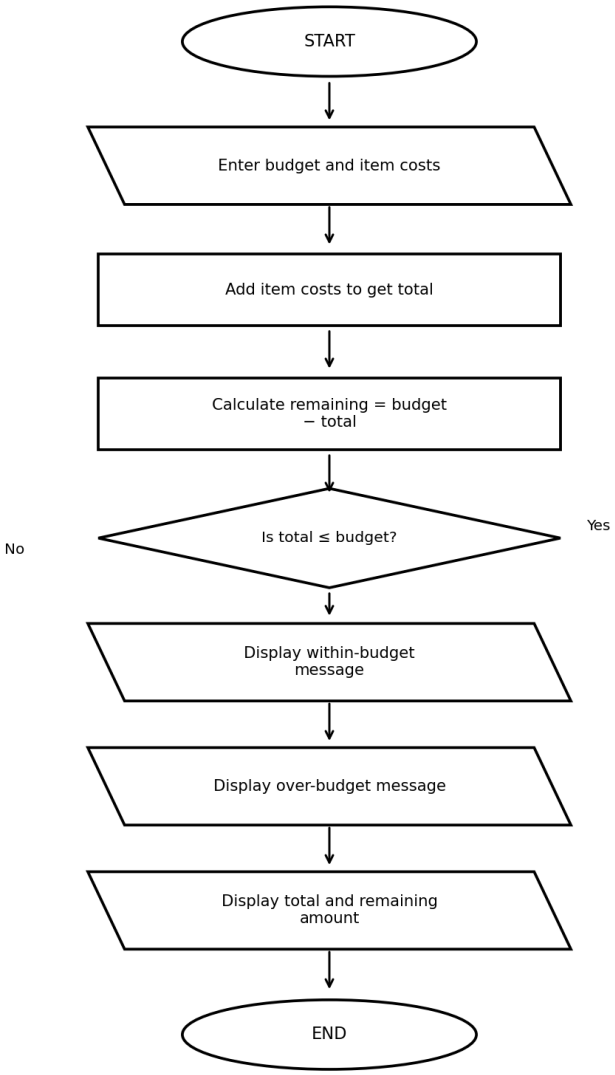
Algorithm

46. Enter budget and number of items.
47. Set total = 0.
48. Enter each item cost and add it to total.
49. If a cost is negative, display an error and stop.
50. Display total.
51. If total is within budget, display money left; otherwise display the amount over budget.

Python Program

```
budget = float(input("Enter budget: ₹"))
n = int(input("Enter number of items: "))
if budget < 0 or n <= 0:
    print("Error: budget must be non-negative and item count positive.")
    raise SystemExit
total = 0.0
for i in range(1, n + 1):
    cost = float(input(f"Enter cost of item {i}: ₹"))
    if cost < 0:
        print("Error: item cost cannot be negative.")
        raise SystemExit
    total += cost
print(f"Total cost: ₹{total:.2f}")
if total <= budget:
    print(f"Money left: ₹{budget - total:.2f}")
else:
    print(f"Amount over budget: ₹{total - budget:.2f}")
```

Flowchart with Standard Symbols



Oval = Start/End | Parallelogram = Input/Output | Rectangle = Process | Diamond = Decision

Pseudocode

```
START
INPUT budget and number of items
total = 0
FOR each item:
    INPUT cost
    IF cost < 0, SHOW error and STOP
    total = total + cost
DISPLAY total
IF total <= budget, DISPLAY money left
ELSE DISPLAY amount over budget
STOP
```

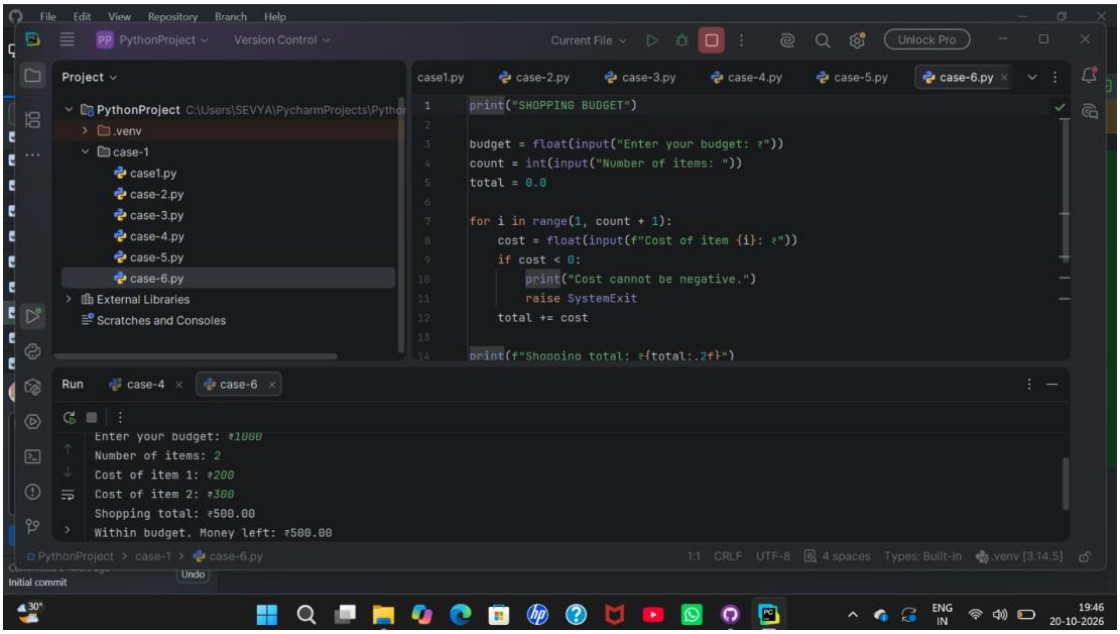
Test Cases

Test	Input	Expected result

Within budget	Budget ₹1000; costs ₹200 and ₹300	Total ₹500; money left ₹500
Over budget	Costs exceed budget	Amount over budget is displayed
Negative cost	Enter a cost below zero	Error message appears

Output Screenshot

The screenshot shows a ₹1000 budget, item costs ₹200 and ₹300, total ₹500.00 and ₹500.00 left.



8. Numeric Pattern Generator

Aim: To print rows of numbers from 1 up to the row number.

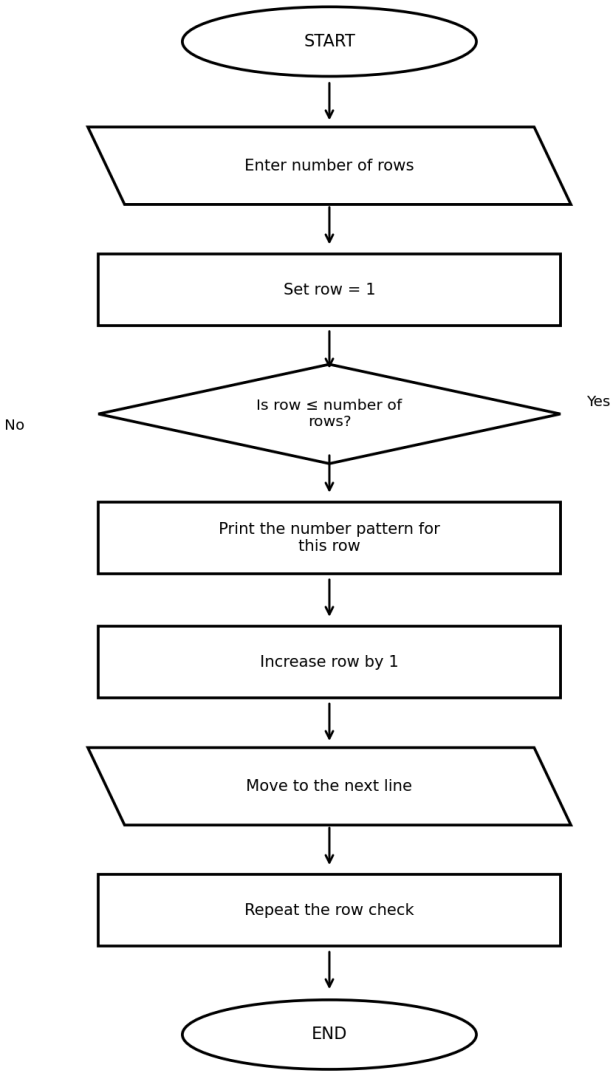
Algorithm

52. Enter the number of rows.
53. If rows is zero or negative, display an error and stop.
54. For each row from 1 to rows, print numbers from 1 to the row number.
55. Move to a new line after each row.
56. Stop.

Python Program

```
rows = int(input("Enter number of rows: "))
if rows <= 0:
    print("Please enter a positive number of rows.")
    raise SystemExit
for row in range(1, rows + 1):
    for number in range(1, row + 1):
        print(number, end="")
    print()
```

Flowchart with Standard Symbols



Oval = Start/End | Parallelogram = Input/Output | Rectangle = Process | Diamond = Decision

Pseudocode

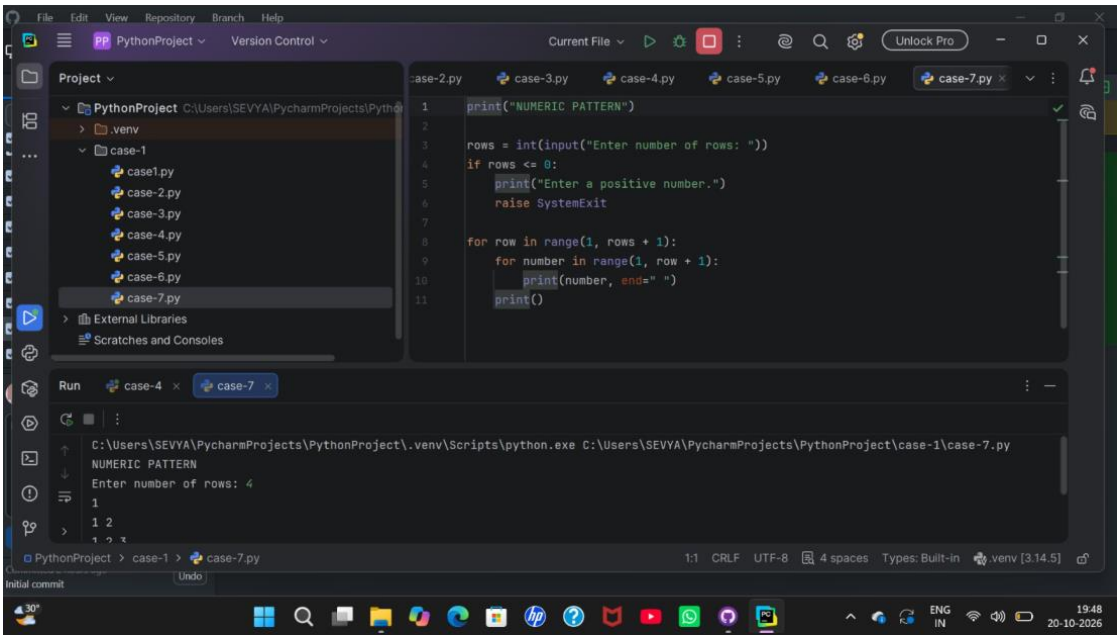
```
START
INPUT rows
IF rows <= 0, SHOW error and STOP
FOR row = 1 TO rows:
    FOR number = 1 TO row:
        DISPLAY number on same line
    MOVE to next line
STOP
```

Test Cases

Test	Input	Expected result
Four rows	Enter 4	1 / 1 2 / 1 2 3 / 1 2 3 4
Zero rows	Enter 0	Positive-number message appears

Output Screenshot

The screenshot shows the Numeric Pattern program with 4 rows; part of the output is visible.



9. Text Compression (Run-Length Encoding)

Aim: To shorten repeated characters by writing each character followed by its count.

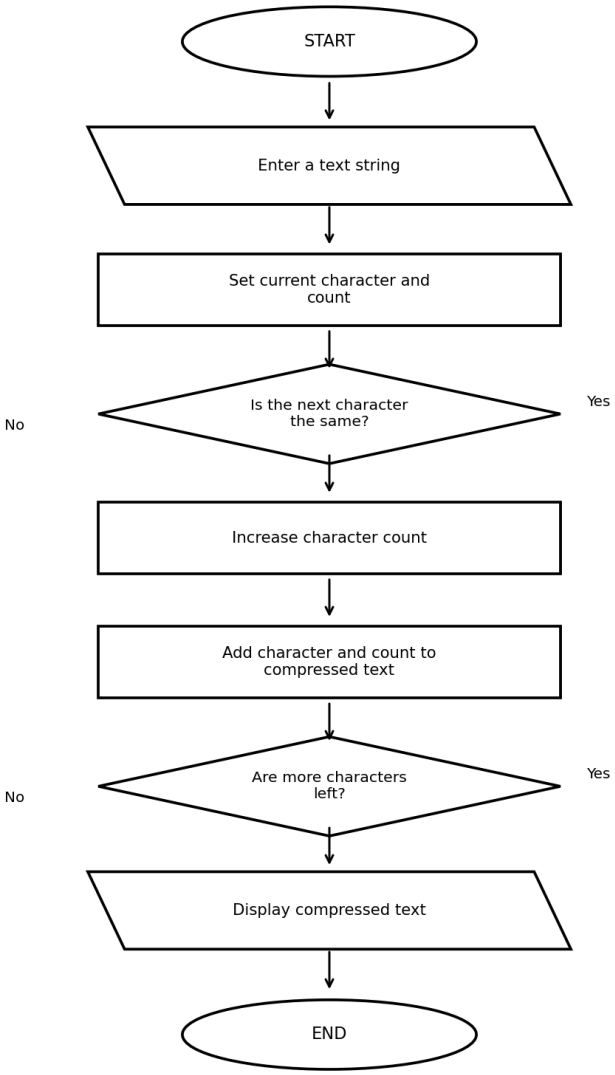
Algorithm

57. Enter a text string.
58. If text is empty, display a message and stop.
59. Set current character to the first character and count = 1.
60. Read the next character. If it matches current, increase count.
61. If it differs, save current character and count, then start counting the new character.
62. Save the last character and count; display compressed text and stop.

Python Program

```
text = input("Enter text: ")
if text == "":
    print("Error: text cannot be empty.")
    raise SystemExit
compressed = ""
current = text[0]
count = 1
for char in text[1:]:
    if char == current:
        count += 1
    else:
        compressed += current + str(count)
        current = char
        count = 1
compressed += current + str(count)
print(f"Compressed text: {compressed}")
```

Flowchart with Standard Symbols



Oval = Start/End | Parallelogram = Input/Output | Rectangle = Process | Diamond = Decision

Pseudocode

```
START
INPUT text
IF text is empty, SHOW error and STOP
SET current = first character; count = 1
FOR each next character:
    IF character = current, count = count + 1
    ELSE save current and count; set new current and count = 1
SAVE final character and count
DISPLAY compressed text
STOP
```

Test Cases

Test	Input	Expected result
Repeated characters	aaabbc	Compressed text: a3b2c1
No repeats	abcd	Compressed text: a1b1c1d1

Empty text	Press Enter without text	Empty-input message appears

Output Screenshot

The screenshot shows input aaabbc and output a3b2c1. The last screenshot in the original set is another view of this program.

